



UNIVERSIDADE FEDERAL DO RIO GRANDE DO NORTE

BACHARELADO EM ENGENHARIA DE SOFTWARE

Linguagem dos Trabalhadores

Fernanda Gonçalves Barros

João Pedro Barreto Mello

Luis Eduardo Fernandes Cândido

Marcos Arthur da Silva Melo

Mário Luiz da Silva Júnior

Natal - RN

Novembro - 2025

Sumário

Sumário.....	2
1. Estrutura dos programas da linguagem proposta.....	3
1.1. Código mínimo compilável.....	3
1.2. Exemplos gerais.....	3
1.2.1. Declaração e Inicialização e atribuição de variáveis.....	3
1.2.2. Atribuições e Expressões.....	4
1.2.3. Declaração e Chamada de função.....	4
1.2.4. Estruturas de controle.....	4
1.2.5. Estruturas de repetição.....	5
2. Estrutura dos analisadores.....	6
2.1. Analisador Léxico:.....	6
2.1.1. Principais tarefas.....	6
2.1.2. Decisões.....	6
2.1.3. Dificuldades.....	6
2.2. Analisador Sintático:.....	6
2.2.1. Principais tarefas.....	6
2.2.2. Decisões.....	7
2.2.3. Dificuldades.....	7
2.2.4. Conflitos.....	8
2.3. Limitações da implementação atual.....	8
3. Uso do analisador sintático.....	9
3.1. Como gerar o analisador sintático.....	9
3.2. Programas de teste e resultados esperados.....	9
3.2.1. Programa de teste.....	9
3.2.2. Resultados esperados.....	13

1. Estrutura dos programas da linguagem proposta

1.1. Código mínimo compilável

O código mínimo compilado precisa ter pelo menos uma função. Atualmente não é feita nenhuma checagem sobre o nome da função e se entre as funções declaradas existe a função principal. Portanto, o analisador consegue compilar com uma função de qualquer nome. Além disso, é necessário que essa função possua um corpo preenchido com instruções. Essas instruções podem ser comandos de entrada, saída, atribuições, chamadas de função, etc. Abaixo está um exemplo de código mínimo compilável.

```
vazio func main() {  
    saida("Olá mundo!");  
}
```

1.2. Exemplos gerais

Também é possível utilizar comandos no escopo global do programa, ou seja, fora de qualquer função. Entre os comandos que podem ser utilizados neste escopo, temos: declaração de variável, inicialização de variável, inicialização de lista e declaração de registro.

Exemplo:

```
int global = 10;  
vazio func main() {  
    saida(10);  
}
```

Abaixo alguns exemplos compiláveis de estruturas já reconhecidas pela Linguagem dos Trabalhadores:

1.2.1. Declaração e Inicialização e atribuição de variáveis

```
vazio func main(){  
    int num;  
    texto titulo;  
    Lista<logico> lista;  
    real i = 3.5;
```

```
    Registro Aluno = {  
        int cpf,  
        texto nome,  
        Lista<int> notas  
    };  
};
```

```
Lista<Aluno> alunos = novo Lista<>();  
Aluno teste;  
alunos[0].nome = "Fernanda";  
alunos[0].notas[0] = 10;
```

```
}
```

1.2.2. Atribuições e Expressões

```
vazio func main(){
    int a = 10 + 10;
    int b = a - 10;
    logico teste = verdadeiro;
    logico errado = falso;
    logico booleano = teste && errado;
    booleano = booleano || teste;
    real expressao_grande = (( a + 10 - 2 ) * 3 ) / b;
    logico comparador = 10 > b;
    a++;
    teste = a == b;
}
```

1.2.3. Declaração e Chamada de função

```
int func soma(int a, int b){
    resultado = a + b;
    retorne resultado;
}
```

```
vazio func main() {
    int resultado = soma(10, 2);
}
```

1.2.4. Estruturas de controle

```
vazio func testarSwitch() {

    int dia = 3;

    saida("Teste 1:");
    escolha (dia) {
        caso 1:
            saida("Segunda");
            pare;
        caso 2:
            saida("Terca");
            pare;
        caso 3:
            saida("Quarta");
            pare;
        padrao:
            saida("Outro dia");
    }
}
```

```

}

vazio func testarlf() {
    int a = 10;
    int b = 5;

    se (a < 5){
        saida("Teste 1");
        se (b > 1) {
            saida("Teste 2");
        }
    }
    senao se (a < 7){
        saida("Teste 3");
        se (b > 1) {
            saida("Teste 4");
        }
        senao {
            saida("Teste 5");
        }
    }
    senao {
        saida("Teste 6");
    }
}

```

1.2.5. Estruturas de repetição

```

vazio func main () {
    int x = 0;
    enquanto(x < 10) {
        saida("somando...");
        x++;
    }

    int y = 0;
    execute {
        saida("executando antes de testar");
        y++;
    } enquanto (y < 10);

    repita(int i=0, i<10, i++) {
        saida("repetindo...");
    }
}

```

2. Estrutura dos analisadores

2.1. Analisador Léxico:

2.1.1. Principais tarefas

O analisador léxico é responsável por ler o código-fonte e convertê-lo em uma sequência de tokens para o analisador sintático. No contexto da nossa linguagem, o analisador léxico reconhece palavras-chave (`func`, `novo`, `adicionar`, `se`, `repita`, `enquanto`, `real`, `Lista`, etc.), operadores (`+`, `-`, `*`, `=`, `&&`, `!=`, `++`, etc.), literais (`ID`, `BOOL_LITERAL`, `INT_LITERAL`, etc.), pontuações (`.`, `“`, `”`, `,`, etc.). Além de ignorar elementos como espaços em branco, quebras de linha e comentários (iniciados com `#`).

2.1.2. Decisões

Como a Linguagem dos Trabalhadores utiliza como base a língua portuguesa, todas as palavras reservadas da linguagem são em português. Apesar disso, os tokens foram criados com nomes em inglês, para ajudar na diferenciação entre token e lexema e também pela familiaridade dos integrantes do grupo com os termos em inglês. Outra decisão importante foi a de não criar tokens para lexemas com apenas um caractere. Ao invés disso, o valor do caractere em ASCII é retornado para o analisador sintático. Com isso, é possível utilizar o próprio lexema encapsulado por aspas simples na definição das regras gramaticais, o que as deixa mais claras e concisas.

2.1.3. Dificuldades

Em geral, não houve muitas dificuldades na etapa de construção do analisador léxico, devido à simplicidade de sua implementação. Contudo, houve algumas divergências na escolha dos tokens utilizados. Por causa disso, algumas mudanças foram realizadas, como a escolha do lexema `“repita”` em vez de `“para”`, `“execute”` em vez de `“faca”` e `“func”` em vez de `“funcao”`.

Além disso, houve dificuldade para definir os literais inteiros e reais, pois expressão regular, não estava permitindo números negativos, e ao tentar corrigir, foi gerado um erro para expressões do tipo aritméticas. Onde o sinal da operação estava sendo reconhecido como sinal do literal. A correção para este problema foi passar essa tarefa para o analisador sintático.

2.2. Analisador Sintático:

O analisador sintático recebe os tokens do analisador léxico e verifica se eles obedecem à Gramática Livre de Contexto (GLC) da linguagem.

2.2.1. Principais tarefas

- **Verificação Estrutural:** Valida se a sequência de tokens forma sentenças válidas de acordo com a gramática.
- **Travessia da Árvore Sintática:** O parser faz o reconhecimento das estruturas gramaticais da linguagem pela travessia da Árvore Sintática Abstrata da linguagem (AST). Como o processo é incremental, o parser vai lendo e

analisando um token de cada vez, não é necessário montar a estrutura da árvore de fato.

- Tratamento de Erros: Invoca `yyerror` quando uma sequência de tokens viola as regras gramaticais.
- Interface para Semântica: Fornece as "ações semânticas" (blocos `{...}` em C) que são executadas quando uma regra é reconhecida. É nessas ações que a análise semântica (verificação de tipos, etc.) deve ser implementada.

2.2.2. Decisões

Nossas decisões para as regras sintáticas foram feitas tendo em mente o objetivo de criar uma linguagem de fácil entendimento para uso educacional de programação de nível básico.

Por causa disso escolhemos ter uma estrutura de bloco obrigatória e bem definida. Estruturas de controle como `if`, `while` e `for` sempre exigirão um bloco de comandos delimitados por `{ ... }`. Esta decisão é crucial, pois *elimina* a ambiguidade clássica do "dangling else".

Seguindo a mesma filosofia também delimitamos bem a finalização das instruções, sendo mantido que será necessário um `;` ao fim de cada instrução, sendo esse caráter guardado exclusivamente para esses casos.

Também decidimos que a gramática exigirá que todo o programa tenha pelo menos uma *func_declaration*, forçando uma estrutura procedural onde teremos sempre uma função principal no programa.

2.2.3. Dificuldades

Houve certa dificuldade para criar as regras que permitiriam acesso aos campos de um registro ou de posições de uma lista. Ambos os tipos de acesso mencionados acima, necessitam de regras recursivas, e além disso, é possível criar uma lista que armazena variáveis de um tipo de registro e declarar uma lista como campo de um registro. Logo, os tipos de acesso poderiam aparecer intercalados em um só comando. Para resolver este problema, foi criada uma regra que unifica o acesso aos campos dessas duas estruturas de dados, a regra `<access>`.

Outra dificuldade encontrada, foram as regras de declaração, inicialização e atribuição de variáveis. Como listas e registros são estruturas de dados mais complexas, existem várias formas de fazer a declaração, inicialização ou atribuição de variáveis desse tipo. Por simplicidade, optamos por remover a regra de inicialização de registro. Sendo necessário inicializar campo por campo.

Além disso, algumas regras geraram conflitos, o que foi uma dificuldade constante durante a execução do trabalho. O detalhamento dos conflitos e como foram resolvidos, está detalhado no item seguinte.

2.2.4. Conflitos

- 2.2.5. Conflito shift/reduce entre `<access>` e `<list_push>` e `<list_remove>`. A regra de acesso, é uma regra que permite que o usuário acesse os campos de um registro ou as posições de uma lista. As regras para adicionar e remover elementos de uma lista, também faziam uso da regra `<access>` seguido de um ``.'`, o que gerou o conflito. A resolução utilizar a regra `<access_suffix_list>` para compor as regras de manipulação de lista, em vez de `<access>`;
- 2.2.6. Conflito shift/reduce do dangling else. Para evitar o conflito do dangling else, já na parte de elaboração da gramática por texto, a regra de `se/senao/senao se`, exige o uso de chaves para marcar o fim de um bloco. Portanto, esse conflito foi um pouco confuso inicialmente, porém a sua resolução se deu ao trocar a recursão da regra `<else_if>` da direita para a esquerda;
- 2.2.7. Conflito reduce/reduce. Devido à possibilidade de haver vazio nas regras `<general_stmt_list>` e `<stmt>`, ocorria um conflito de reduce/reduce quando o programa chegava nessa determinada situação que podia reduzir para os dois;
- 2.2.8. Conflitos reduce/reduce e shift/reduce. Ao montar as regras de expressões, foi utilizada a regra `<expression>` sem o parênteses na regra `<unary>`. Como isso fazia com que a regra ficasse circular, foram gerados muitos conflitos reduce/reduce e shift/reduce. Porém o problema foi resolvido ao adicionar os parentes encapsulando o `<expression>` na regra de `<unary>`;

Na implementação final, não há conflitos do tipo shift/reduce, pois todos os conflitos foram tratados.

2.3. Limitações da implementação atual.

Alguns dos itens propostos pela linguagem ainda não foram implementados no analisador sintático, devido ao curto prazo de tempo. Entre eles estão:

- Conversão de tipos primários;
- Expressões com NOT;
- Não é possível inicializar listas com exemplo de lista completa;
- Não é possível remover elementos específicos de uma lista.
- Funções vazias;
- Comentários de bloco;

A implementação da nossa linguagem foca estritamente no léxico-sintático. O parser atualmente não está gerando nenhum código, então ao rodar ele só está reconhecendo as regras porém nenhuma árvore sintática ou código intermediário é gerado. Assim ela não é um compilador funcional. A ausência da análise semântica impacta em diversos campos, entre eles impede o analisador de realizar as verificações de:

- Tipos: Permitindo uma variável receber qualquer valor independente do seu tipo declarado.
- Declarações: assim é possível chamar uma variável não declarada.
- Duplicidade: não é impedido que duas variáveis possuam o mesmo nome no escopo.

O tratamento de erros é bastante limitado também, usando o sistema padrão do Bison com o “yyerror” que interrompe a análise no primeiro erro de sintaxe.

3. Uso do analisador sintático

3.1. Como gerar o analisador sintático

Para compilar e executar o analisador léxico-sintático a partir dos arquivos-fonte (lexico.l, sintatico.y), é necessário ter o Flex, o Bison e um compilador C instalados.

O processo de *build* segue os seguintes passos em um ambiente de terminal:

- **Gerar o Parser (Bison):** O comando bison processa o arquivo de gramática. A flag -d é crucial, pois gera o arquivo de cabeçalho (.tab.h) que define os tokens para o analisador léxico.

```
bison -d sintatico.y
```

- **Gerar o Scanner (Flex):** O Flex processa o arquivo de análise léxica. Ele incluirá o sintatico.tab.h gerado anteriormente.

```
flex lexico.l
```

- **Compilar e Linkar (GCC):** Os arquivos C gerados são compilados e linkados em um único executável.

```
gcc lex.yy.c -o lexer
```

- **Execução:** O compilador pode então ser executado, recebendo um arquivo de código-fonte da linguagem proposta como entrada padrão (stdin).

```
./lexer < codigo_fonte.txt
```

3.2. Programas de teste e resultados esperados.

3.2.1. Programa de teste

função recursiva que particiona a lista em 2 partes

```
vazio func ordenar(Lista<int> a, int inicio, int fim) {
```

```
    se (inicio < fim) {
```

```
        meio = ( inicio + fim );
```

```
        ordenar(a, inicio, meio);
```

```
        ordenar(a, meio+1, fim);
```

```
        mesclar(a, inicio, meio, fim);
```

```
    }
```

```
}
```

função que une duas listas criando uma lista ordenada

```
vazio func mesclar (Lista<int> a, int inicio, int meio, int fim){
```

```

Lista<int> aux = novo Lista<>();

# copiando trecho da lista que vai ser ordenada
repita (int i = inicio, i < fim, i++){
    aux.adicionar(a[i]);
}

int i = inicio;
int j = meio + 1;
int k = inicio;

# junção e ordenação das duas listas
enquanto ( i <= meio && j <= fim){
    se (aux[i] < aux[j]){
        a[k] = aux[i];
        i++;
    }
    senao {
        a[k] = aux[j];
        j++;
    }
    k++;
}

# adiciona os itens que não foram adicionados da primeira lista
enquanto ( i <= meio ) {
    a[k] = aux[i];
    i++;
    k++;
}

# adiciona os itens que não foram adicionados da segunda lista
enquanto ( j <= meio ) {
    a[k] = aux[j];
    j++;
    k++;
}

# Função principal que aplica o algoritmo Quicksort recursivamente.
vazio func ordenacao_rapida(Lista<int> a, int inicio, int fim) {
    # O caso base da recursão: continua apenas se houver mais de um elemento para
    ordenar.
    se (inicio < fim) {
        # Chama a função para particionar a lista e obter o índice final do pivô.
        int pivo_indice = particionar(a, inicio, fim);
    }
}

```

```

    # Chama recursivamente a função para a sub-lista à esquerda do pivô.
    ordenacao_rapida(a, inicio, pivo_indice, - 1);

    # Chama recursivamente a função para a sub-lista à direita do pivô.
    ordenacao_rapida(a, pivo_indice + 1, fim);
}
}

# Função que particiona a lista, colocando os elementos menores que o pivô à sua
esquerda e os maiores à sua direita. Retorna o novo índice do pivô.
int func particionar(Lista<int> a, int inicio, int fim) {
    # Estratégia simples: escolher o último elemento como pivô.
    int pivo_valor = a[fim];

    # 'i' será o índice do último elemento que é menor que o pivô.
    # Começa antes do primeiro elemento.
    int i = inicio - 1;

    # Percorre a lista do início até o penúltimo elemento.
    repita (int j = inicio, j < fim, j++) {
        # Se o elemento atual for menor ou igual ao pivô...
        se (a[j] <= pivo_valor) {
            # ...incrementa 'i' e troca o elemento em 'i' com o elemento em 'j'.
            i = i + 1;
            trocar(a, i, j);
        }
    }

    # Ao final, coloca o pivô em sua posição correta, trocando-o com o elemento que está
    logo após o último elemento menor que ele.
    trocar(a, i + 1, fim);

    # Retorna a posição final do pivô.
    retorne i + 1;
}

# Função auxiliar para trocar dois elementos de posição na lista.
vazio func trocar(Lista<int> a, int pos1, int pos2) {
    int temp = a[pos1];
    a[pos1] = a[pos2];
    a[pos2] = temp;
}

vazio func testarSwitch() {

```

```

int dia = 3;

saida("Teste 1:");
escolha (dia) {
    caso 1:
        saida("Segunda");
        pare;
    caso 2:
        saida("Terca");
        pare;
    caso 3:
        saida("Quarta");
        pare;
    padrao:
        saida("Outro dia");
}

int valor = 1;
saida("Teste 2:");
escolha (valor) {
    caso 1:
        saida("Caso 1");
    caso 2:
        saida("Caso 2");
        pare;
    caso 3:
        saida("Caso 3");
        pare;
}

int x = 10;
saida("Teste 3:");
escolha (x) {
    caso 10:
        saida("Dez");
        pare;
    caso 20:
        saida("Vinte");
        pare;
}
}

vazio func main() {
    #testando registro e lista
    Registro reg = {
        int cpf,

```

```

        texto nome,
        reg2 r,
        Lista<reg> lista
    };

    continue;
    reg r;
    r.cpf = "valor";
    r.nome = "oi";
    r.r.cpf = 1312321;
    r.lista[0] = r;
    r.lista[0].cpf = "oie";

    Lista<logico> teste;
    teste = novo Lista<>();
    teste.adicionar(verdadeiro);
    logico a = teste[0];
    teste[1] = a;

    Lista<reg> outro_teste = novo Lista<>();
    teste[0].cpf = a;
    teste[1].lista[0].cpf = a;
    r.lista[0] = a;
    lista.remove();
    lista.adicionar(a);
}

```

3.2.2. Resultados esperados

Ao processar o programa de teste, é esperado que a análise seja concluída sem a emissão de qualquer mensagem de erro. Esse resultado indica que a sequência de tokens do código-fonte obedece à gramática estabelecida na linguagem.

Caso houvesse violações às regras de gramática, seria esperado que o analisador parasse no primeiro erro encontrado e imprimisse uma mensagem de erro informando o número da linha onde ocorreu o erro e o token que causou a falha, o que, felizmente, não foi o caso.